OpenSSL CONFERENCE

Hardware-enforced cryptographic contexts that RISC-V software can use but never read

KLE — the RISC-V Cryptographic Lockers Extension (Zkl)

Speaker: **Radim Krčmář** · Qualcomm · RISC-V High Assurance Cryptography Task Group

Authors: Roberto Avanzi (lead), Ruud Derwig, Luis Fiolhais, Radim Krčmář

§ PRIVACY AS A FUNDAMENTAL HUMAN RIGHT

15.10.2026

OpenSSL CONFERENCE

Where is your AES key right now?

Yours: in the heap of any library, as long as the context lives.

Not yours to hold: content, storage and attestation keys, used only across a TEE, HSM or kernel boundary.

15.10.2026

```
EVP_CIPHER_CTX
 └─ algctx ──► PROV_AES_CTX
                 └─ ks.ks : AES_KEY
                      rd_key[60]   // expanded key
                      rounds
```

- AES-NI, RISC-V Zkn and Zvkned speed up the rounds — the round keys are still loaded from memory on every call.
- The key schedule is invertible: read it, and you have the key. One memory-disclosure bug, DMA attack or cold-boot read away.

Where a key can live, and what it costs

	What it gives	What it costs
Crypto instructions	No boundary crossings. Well known.	Keys can leak.
TEE, HSM	Keys are safe. Discipline enforced.	Latency, expensive round trips. Cumbersome to configure.
Key-hiding ISAs	Software sees only a wrapped key. IBM protects some mode state as well.	Instruction creep. Intel limited to AES. Key not bound to mode of operation.
KLE's goal	In-process. All state hidden. Single interface for many algorithms. Key can reach you already sealed.	No silicon or numbers yet, nonce discipline is yours

Hiding the key is not enough

Partial operations

4–5 rounds of AES suffice to recover the key.

Even AES in two calls is too much.

Derived secrets

GCM's H, XTS masks, GCM-SIV keys: leak them and the mode breaks.

A key not bound to its mode gives them away: run it in ECB to get H.

Scale

Instructions per primitive don't scale to SM4, Ascon, ML-KEM...

Each algorithm adds opcodes, key formats and OS support.

What KLE wants to achieve

OpenSSL CONFERENCE

HSM-grade security at CPU-instruction speed.

- **Confidentiality and integrity** of keys and derived secrets
- **Many algorithms**, one interface, combined in hardware
- **State machines enforced**: an algorithm runs only as its standard allows
- **Scalability**: one ISA from microcontrollers to servers, contexts that migrate
- **Few, simple instructions, minimal hardware**: a new algorithm adds no instruction

Shape it while it can still change

KLE — KLEE in the programme — is at v0.6, “Developed”: assume everything is subject to change.

15.10.2026

01 Lockers, atomic execution, algorithms

02 Sealed contexts and policy bindings

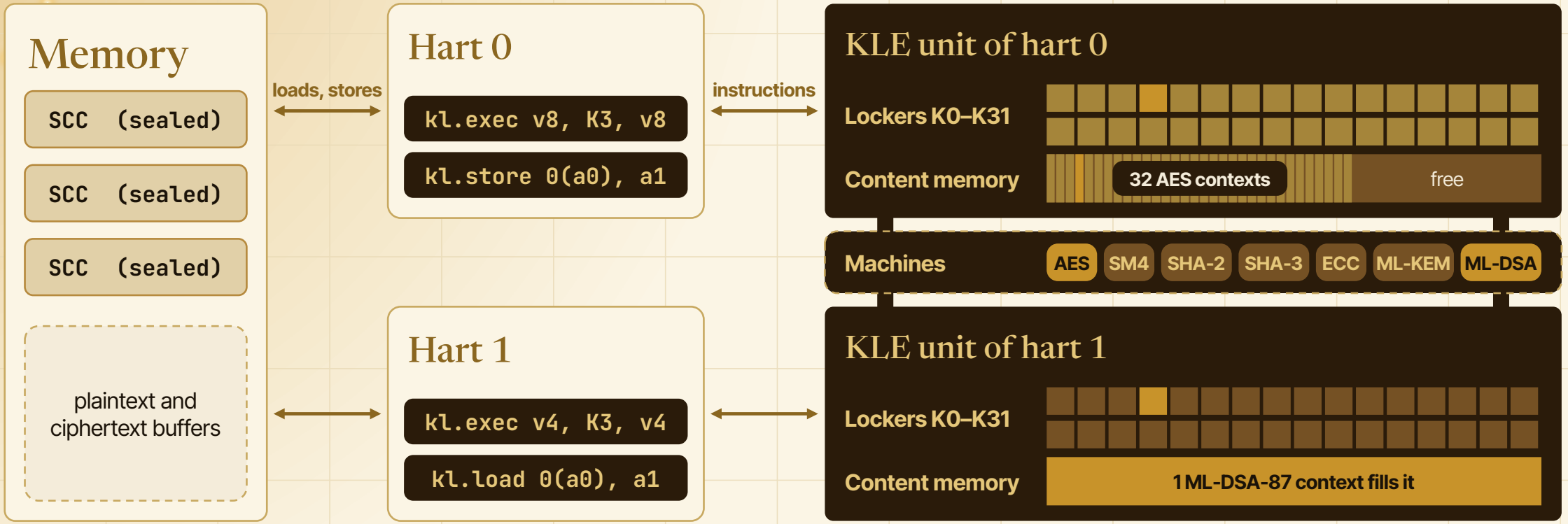
03 A prototype, and what software will decide

04 Before the freeze — your feedback

The ask: tell us where this breaks – for your library, HSM or key server.

KLE in one picture

OpenSSL CONFERENCE



Never a key in clear

The only path to memory

No path to memory

Software names lockers, never keys.

A locker holds a Cryptographic Context

OpenSSL CONFERENCE

always readable · `kl.getmd` | never readable by software



Machine, policies, State, Localities, expiry

keys, derived keys, IV or nonce, intermediate state

implementation-specific data

32 per hart

K0–K31, named by a 5-bit immediate; the same lockers in every privilege mode.

Empty or configured

An Unconfigured locker holds nothing; its MDH reads zero.

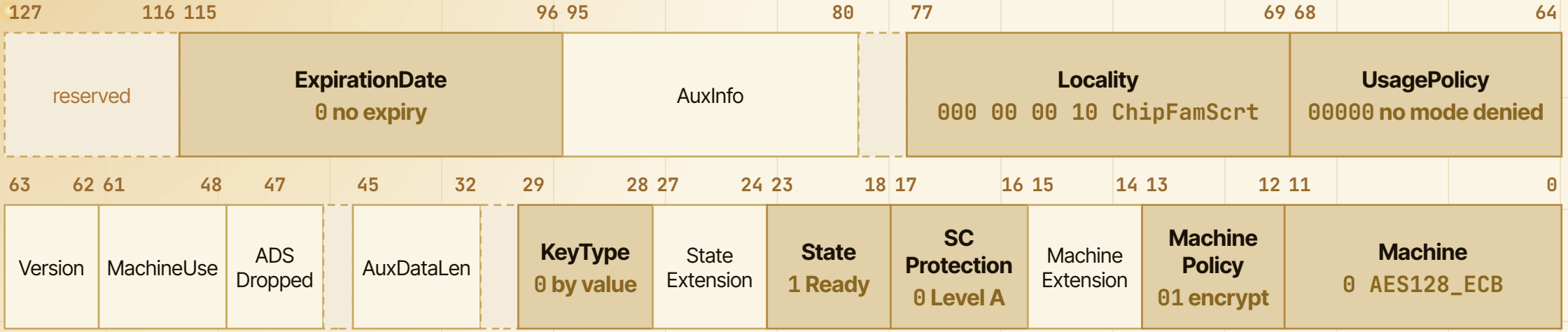
Capacity is finite

Content lives in the KLF, whose size is implementation-defined; running out raises an exception at provisioning or import.

The MDH: everything but the key

OpenSSL CONFERENCE

Locker K3, an encrypt-only AES-128 key bound to the chip model. **Highlighted:** the fields this talk uses.



- **Read it.** `kl.getmd` returns it at any time; `kl.getst` extracts the State, e.g. after an AEAD.
- **Build one.** Provisioning starts from an MDH; `kl.avail` and `kl.size` take one to report support and SCC size.
- **Trust it.** The first 16 bytes of every SCC: in the clear, but authenticated — alter a policy and import fails.

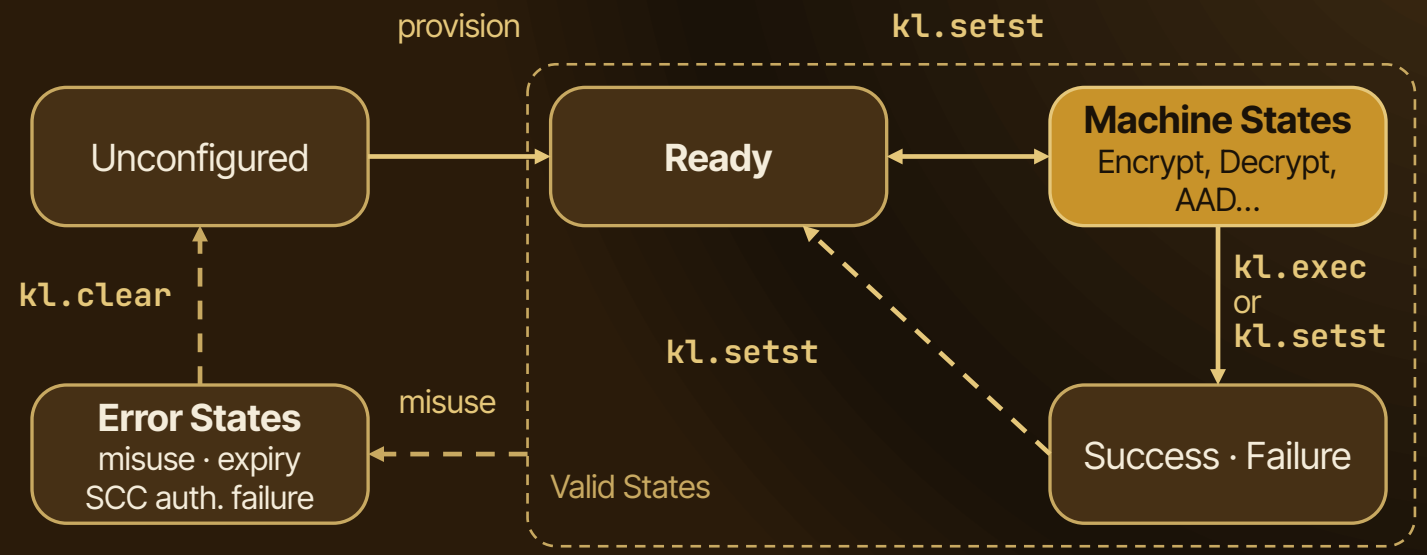
OpenSSL CONFERENCE

Atomic execution

`kl.setst` moves the State.

`kl.exec` processes whole blocks only.

15.10.2026



- Interruptible only between blocks, resuming at `klstart` (like `vstart`).
- Long operations — scalar multiplication, key generation, signing — keep their progress inside the unit.
- Error States are sticky and silent: later use outputs zeros, without a trap.

Same binary, different cipher

```
IMPORT   K0, (a5)   // SCC → K0: AES-128-GCM, decrypt-only
kl.setst K0, #kl_state_set_aux_value, a0   // a0 = 96: IV bits
kl.exec  K0, v1     // nonce = iv ^ seq → J0 inside
kl.exec  K0, v2     // AAD: the record header, zero-padded
kl.setst K0, #kl_state_decrypt
loop:               // whole blocks: absorb, decrypt
  kl.exec  v0, K0, v0
  …
kl.setst K0, #kl_state_dec_last_block, a1  // a1: bits left
kl.exec  v0, K0, v0
kl.setst K0, #kl_state_dec_tag_finalize, v3  // v3: lengths
kl.setst K0, #kl_state_hash_verify, v4  // v4: the received tag
kl.getst a0, K0     // Success or Failure: only then use PT
```

The opcode never names the algorithm. The Metadata does.

- AES-128-GCM, AES-256-GCM or SM4-GCM context: same binary.
- Decrypt-only key: the read side can never enter Encrypt, so it never makes a tag.
- Data in vector registers (`Zklv`).

Supported algorithms (Machines)

OpenSSL CONFERENCE

AES-128 · 192 · 256
ECB · CTR · XTS · GCM · …

SM4
ECB · CTR · XTS · GCM · …

SHA-2
All supported

SHA-3
All supported

Ascon
All SP 800-232 algorithms

SM3 · BLAKE2
Both supported

Elliptic curves
NIST · brainpool · SM2 · …

Post-quantum
ML-KEM · ML-DSA

Mandatory for Zkl
AES in ECB, GCM and GCM-SIV modes, and SHA-2.
The rest is optional; `kl.avail` discovers it.

Left to software
CBC, CFB and OFB are built on ECB: none of their intermediates needs secrecy.

Policy travels with the key

What

MachinePolicy
`MDH[13:12]`

Fixed Machine; encrypt, decrypt or both.

Who

UsagePolicy
`MDH[68:64]`

Deny U, VS, S or M; allow Debug.

Where

Locality
`MDH[77:69]`

Chip, device, boot, VM or process.

Until

ExpirationDate
`MDH[115:96]`

Optional: hours since 2027, on a secure clock.

How well

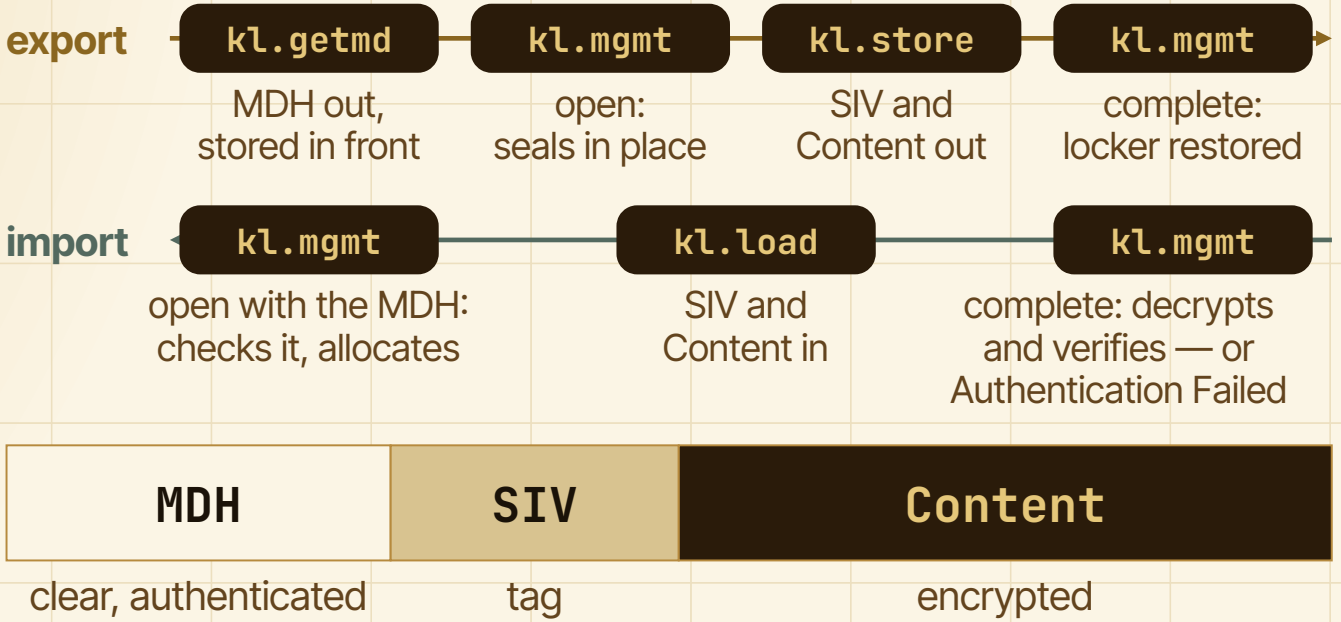
SCProtection
`MDH[17:16]`

A: constant time
B: + differential
C: + fault detection

Sealed contexts

An SCC is a locker's context under authenticated encryption. A process given an SCC never holds the key.

15.10.2026



- Sealed by a variant of AES-256-GCM-SIV under the Context Sealing Key (CSK).
- Bound to a device, boot or process: imports nowhere else.
- No software sees the CSK — unless M-mode sets it, and can then unseal every SCC.

Binding contexts to hardware and software

OpenSSL CONFERENCE

SoC
`Locality[1:0]`

SiPScrt
SoC designer

or

ChipFamScrt
chip model

or

ChipScrt
this chip

+

Device
`Locality[3:2]`

OEMScrt
OEM

or

ProdScrt
product

or

DevScrt
this device

+

Boot
`Locality[5:4]`

PhysBootScrt
host boot · set by M-mode

or

VirtBootScrt
VM boot · set by the hypervisor

+

Software
`Locality[8:6]`

MLocality
supervisor domain
set by M

+

HLocality
virtual machine
set by H

+

SLocality
process
set by the OS

Getting keys in: provision, generate, derive

OpenSSL CONFERENCE

Provision	Generate / System key	Derive
Already sealed Trusted software or an HSM seals it for this device; you import the SCC	**Random key** You provision with no key; the unit generates one	**Shared secret** `kl.derive` moves a DH or KEM shared secret into a symmetric algorithm's key field
or	*or*	↓
Your own key You provision a locker with it, then delete your copy	**System key** You provision with its name; the system key store supplies the key	**Keeps its constraints** Policies narrow; SCProtection can't drop

Does it work? A prototype.

End to end, for AES: ECB, CTR and GCM · functional, not timed

+5.4k

QEMU

The KLE unit: lockers, AES Machines, sealing

`github.com/radimkrcmar/qemu`

+1.7k

Linux

Lockers that follow the task

`github.com/radimkrcmar/linux`

+5.6k

OpenSSL

Two providers: transparent, and opaque `EVP_SKEY`

`github.com/radimkrcmar/openssl`

Branch `zkl` of each · build and test all three with `nix run github:radimkrcmar/kle-demo`

It works: a key you never had

OpenSSL CONFERENCE

The provisioning lane, live: the host seals a key under the CSK; the guest decrypts with the SCC.

```
host$ nix run github:radimkrcmar/kle-demo#provision -- Hello OpenSSL 2026!
CSK (QEMU gets it as x-zkl-csk):     f1caf1c7b5c906c7…6023dab2f5446b12
AES-256 key (stays here, sealed):    e9218c9991ed4ade…0dfafcd0465d3949
guest$ xxd scc
00000000: 2020 0400 0000 0000 0e00 0000 0000 0000    // the MDH
00000010: fb7a de96 9c68 6758 fde1 efd2 afbc 832f    // the SIV
00000020: 414b fed6 7faf b738 7625 4bb5 596b e34c    // the sealed key
00000030: d803 4e8f cdbe 0e28 b886 c128 838b 37e9
guest$ xxd message.enc
00000000: 05bd 03cc 4baa 3d69 11e2 8e01 b358 1f6f    // the IV
00000010: 9d39 b1c2 cc8e 64ef 9d15 45b2 93c9 616b    // the ciphertext
00000020: 496a d5d1 44d6 48c4 8ca0 b7d1 3e24 fb95
guest$ tail -c +17 message.enc | openssl enc -d -aes-256-cbc -provider kle -skeyuri kle:scc \
    -iv $(head -c 16 message.enc | xxd -p)
Hello OpenSSL 2026!
```

The guest never has the key: the kle provider loads the SCC as an opaque **EVP_SKEY**.

The same SCC refuses to encrypt, fails with one bit flipped, and imports nowhere under another CSK.

What the OS and hypervisor will decide

OpenSSL CONFERENCE

Lockers are architectural state: they switch with the task.

Fixed in the ISA · freezes at v0.9	Left to software · can change in a patch
Per-locker Dirty status	→ **Kernel**: when to save and restore
SLocality: one per process	→ **OS**: what fork and exec keep
An SCC imports only under its CSK	→ **Hypervisor**: how a VM's CSK moves

What libraries will decide

OpenSSL CONFERENCE

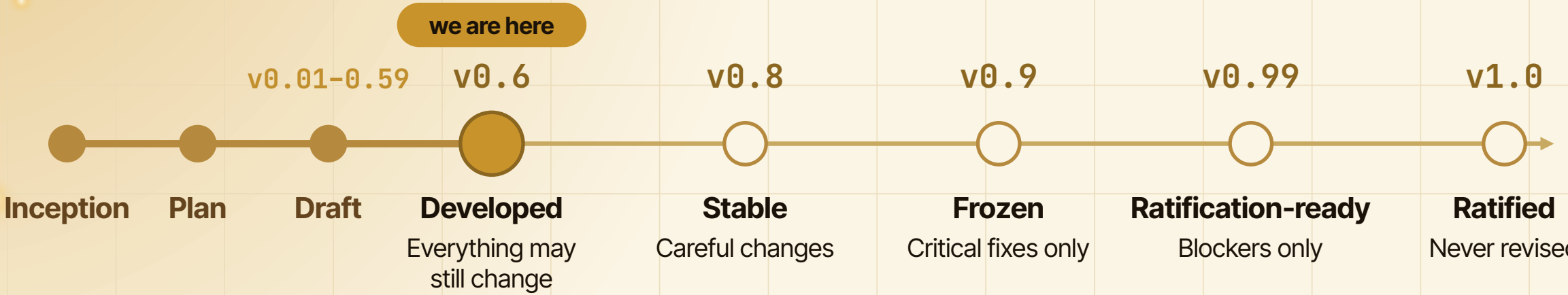
Everything in a thread shares the 32 lockers and the KLF behind them.

Fixed in the ISA · freezes at v0.9	Left to software · can change in a patch
A locker doesn't say whose key	→ **Library or ABI**: who uses which locker
One big key can fill the whole KLF	→ **Library or ABI**: who makes room when it's full

Software can change these answers in a later patch. The instructions underneath can't.

How a RISC-V extension becomes ISA

OpenSSL CONFERENCE



This is the perfect moment for the OpenSSL community to help shape the ideal KLE for their needs.

Questions for you

Algorithms

No RSA, ChaCha20-Poly1305 or AES-CCM.

Which would stop you?

Derived keys

Every HMAC output in a **`kl.derive`** chain stays readable.

Should MACs be derive-only?

Capacity

How many keys does a busy thread use at once, and how big?

Should a minimum be guaranteed?

“The key goes in.

It never comes out.”

Thank you — questions?

Feedback: here, in the hallway, or on GitHub.

Radim Krčmář · Qualcomm · github.com/riscv/riscv-high-assurance-cryptography

The prototype: github.com/radimkrcmar/kle-demo

Backup

Specification facts as of c6ada5683d69 (13 Oct 2026)

Backup · Who does what

OpenSSL CONFERENCE

Role	Responsibilities	Sees keys?
KLE unit	Runs the Machines; enforces policies; seals and unseals; keeps the KLF confidential and intact	Inside only
M-mode firmware	Programs the CSK, or has a secure block set it, unless hardwired or boot-random; PhysBootScrt, MLocality; may emulate KLE	Only via CSK or emulation
Hypervisor	Tags VMs (HLocality, VirtBootScrt); switches guest KLE state	No
OS kernel	SLocality per process; saves, restores and clears lockers; defines thread / fork / exec behaviour	No
Trusted provisioning	TEE applet, HSM, key server or secure block: obtains keys and policies, provisions contexts, revokes	Yes
Crypto library	Allocates lockers†; SCCs as key handles†; nonces, padding, digest reduction; checks State	No*
Application	Chooses policies and bindings; stores SCCs, never keys	No*

* unless the application provisions a raw key through the library † the prototype's choice: the spec has no locker allocation or library model

Backup · The fine print

KLE protects

- Confidentiality and integrity of keys and critical derived values
- Against compromised processes and attackers who read or tamper with memory (DMA, bus interposers)
- The state machine: no skipped or reordered steps, no partial blocks

KLE accepts, or leaves to others

- Plaintext in the process; memory corruption of that process; release of unverified plaintext
- A compromised process can still use its own contexts; a compromised kernel or firmware, any it can import
- Revocation, a key's availability, full key management
- Invasive attacks (FIB/SEM); side-channel protection beyond constant time is optional
- Replay: re-importing an SCC restores its state — nonce discipline stays in software

"Never read" holds outside authenticated debug, M-mode too, unless it programs the CSK or emulates the Machine. A CTR keystream or ECDH secret is an output by design, unless `kl.derive` keeps it inside.

Backup · Instructions

OpenSSL CONFERENCE

Instruction	What it does
`kl.exec`	One complete unit of cryptographic work, as the State defines; Forms A–D (vector operands or KLIOBUF)
`kl.setst`	Moves the Machine between States; kl.clear, kl.clearall, kl.clearads are pseudo-instructions
`kl.mgmt`	Opens or completes provisioning, export or import
`kl.getnl / kl.setnl[i]`	Read or set the named locker, which `KLNL` names in place of any locker operand, kl.exec included
`kl.load / kl.store`	Transfer a locker image to or from memory (aliases kl.input / kl.output use the KLIOBUF)
`kl.mvin / kl.mvout`	Move image blocks through GPRs, vector registers or the KLIOBUF (Zklmv)
`kl.getmd[l|v]`	Read the MDH; kl.getst / kl.getstx extract the State
`kl.size / kl.avail`	Image size; whether a Machine / policy / KeyType / SCProtection combination exists
`kl.restrict[l|h|v]`	Narrow MachinePolicy (a set bit removes a function), Locality, UsagePolicy, ExpirationDate
`kl.clone / rename / swap`	Copy, move or exchange contexts between lockers
`kl.derive`	Move a secret output into another locker without exposing it

Encodings are provisional (MISC-MEM funct3 3 and 4, spare LOAD/STORE funct3); the ARC allocates them.

Backup · Metadata Header (MDH)

OpenSSL CONFERENCE

Bits	Field	Meaning
[11:0]	Machine	Primitive, mode or protocol; ≥ 3072 custom
[13:12]	MachinePolicy	Encrypt / decrypt (sign / verify) permissions
[15:14]	MachineExtension	Reserved to extend Machine and policy
[17:16]	SCProtection	Side-channel Level A–C (0–2); 3 reserved
[23:18]	State	0 Unconfigured, 1 Ready, 2–45 Machine, 46/47 Success/Failure, 48–55 Error, 56–63 Configuration
[27:24]	StateExtension	Machine-specific extra state
[29:28]	KeyType	0 by value, 1 system key (SKID), 2 random at provisioning (PI only)
[45:32]	AuxDataLen	Length of the implementation-specific ADS
[47]	ADSDropped	ADS dropped, layout kept (unauthenticated hint)
[61:48]	MachineUse	Machine- or implementation-specific
[63:62]	Version	MDH format version
[68:64]	UsagePolicy	Deny U / VS / S / M; grant Debug
[77:69]	Locality	Bound Locality Secrets
[95:80]	AuxInfo	Machine- or implementation-specific
[115:96]	ExpirationDate	Hours since 2027-01-01 UTC (Zklexpire)

Backup · KLE exceptions

OpenSSL CONFERENCE

Exception	Raised when
`fatal-error`	Unrecoverable failure in the KLE unit; KLE state reset
`missing-CSK`	No Context Sealing Key, or the mklcsk group only partially written
`locker-Off`	Access to a locker whose *llockerstatus is Off — lazy loading, trap-and-emulate
`unsupported-Metadata`	Unsupported Metadata when opening a provisioning or import
`privilege-violation`	A usage instruction that UsagePolicy forbids, or the use, cloning or restriction of a locker under management
`clock-unavailable`	Zklexpire: the secure clock cannot be read; State unchanged
`out-of-memory`	Insufficient KLF capacity at provisioning, import or clone
`RBG-unavailable`	Random bits unavailable past an implementation-defined bound; State unchanged

In decreasing priority among KLE causes; codes TBD by the ARC.

Backup · Context Sealing Key

OpenSSL CONFERENCE

M-mode

Programs it through the `mklcsk` CSRs.

The only way software sees the CSK, and so every sealed key.

Secure block

E.g. Caliptra, on M-mode's request.

No software, M-mode included, sees it.

Hardwired

Per die or per batch.

Can't be reconfigured or revoked.

Boot RBG

Random at boot, invisible to software.

Shared by the subsystem's harts; SCCs don't survive power loss.

Shared across devices? Device-bound contexts must name ChipScrt or DevScrt.

Backup · Side channels and certification

OpenSSL CONFERENCE

Level A

SCProtection = 0 · constant time

The same operation on the same number of input units takes the same time, whatever the data.

Level B

SCProtection = 1 · + differential protection

Level A plus differential side-channel protection.

Level C

SCProtection = 2 · + fault detection

Level B plus detection of injected faults: the correct result, or a fatal error.

Same code at every level

- Level A is mandatory; the level lives in the MDH, and `kl.avail` reports which a Machine has
- Each level offers at least the guarantees of the one below
- A derived shared secret keeps at least its source's level
- KLF, sealing and CSK paths should match the locker's level

Documented, not defined, by the spec

- Per level: standard and edition, assessment profile, security level, test metrics
- ISO/IEC 19790, 24759:2025, 17825:2024; FIPS 140-3 — SP 800-140F lists no non-invasive metrics yet
- Test effort is not resistance time: a FIPS level states no attack duration. Providers negotiate level and profile

Open question for a FIPS provider on KLE: which duties should the unit carry itself — known-answer tests, pairwise consistency tests on generated key pairs, a service indicator?

Backup · Prototype results

303

QEMU checks

Bare metal; 262 without the CTR Machine, 251 without GCM; SCCs byte-identical to the spec's model

316

AES KATs

OpenSSL vectors, no errors; ECB/CBC/CTR and GCM run in lockers; 51 + 157 in the kle provider alone

29

zkl_test checks

Linux: threads on 4 harts and on one; fork, exec; preemption; lazy loading

34 / 34

kle_test cases

EVP_SKEY; threads, one hart; fork; 40 contexts; CTR and GCM mid-stream; stale records

Functional only, in QEMU TCG, with every Machine and with ECB alone. 3 spec commits behind this talk.

Backup · Inside the kle provider

OpenSSL CONFERENCE

```
/* kl.exec v8, KLNL, v8 over n bytes */
old = kle_nl_push(k);  // kl.getnl, kl.setnl
vsetvli  vl, n, e8, m8, ta, ma
vle8.v   v8, (in)
.4byte   KLE_R4(KLE_F7_KLNL, 8, 0, 8)
vse8.v   v8, (out)
kle_nl_pop(old);       // kl.setnl
```

```
/* ctr128_f on the CTR Machine */
synced = held_gen == k→gen; /* no reload */
if (!synced || memcmp(held_iv, iv, 12))
    kle_ctr_set_iv(k, iv);   /* new IV */
if (!synced || held_ctr ≠ c)
    kle_ctr_set_ctr(k, c);   /* jump */
kle_ctr_xor(k, in, out, 16 * nb);
memcpy(held_iv, iv, 12);
held_ctr = c + nb; held_gen = k→gen;
```

Per-thread ownership

- A locker belongs to a thread; the SCC is the real key handle
- `kle_key_ready()` re-imports it on another thread, or after a clear
- One table per thread, shared by both providers; each evicts only its own
- Every instruction names its locker as `KLNL`: no 32-way switch, but every call leaves KLS Dirty
- `include/crypto/riscv_kle.h`, `cipher_aes_hw_kle.c` (prototype)

Backup · Performance: what to expect

OpenSSL CONFERENCE

What the design implies

- No round keys in memory: nothing to load from the heap per call
- One `kl.exec` per vector register group: LMUL up to 8, 128 B per instruction at VLEN 128
- GCM and GCM-SIV run whole, tag included: no software GHASH
- In-process: no TEE or HSM round trip

What decides the cost

- An attached unit: every instruction is a message over a coprocessor interface
- Several harts may share one unit
- Context switch: export and import of Dirty lockers — ≥ 3 instructions each way, more if `kl.load` / `kl.store` trap
- AES-128 SCC: 48 B without an ADS, ≥ 80 B with one
- Per-locker status: export only Dirty lockers, import lazily

Not timed yet

- QEMU TCG only counts: 16 lockers, 1 changed per slice → 2 SCC operations per switch, not 32
- Cycles per byte against `Zvkned` on hardware
- Time of a context switch, and of a locker-Off exception
- A shared unit under load

The design aims at in-process speed; the numbers have to come from silicon.

Backup · Open items for the ARC

OpenSSL CONFERENCE

Item	Status
Opcodes	Core instructions in MISC-MEM; kl.load / kl.store in the spare LOAD / STORE funct3 — provisional
misa.L, KLS	Provisional and contested; the spec needs only a two-bit FS/VS-like status field
CSR addresses	Placeholders, and so is the Smstateen bit
Exception causes	Eight causes, codes TBD; the locker number in mtval would spare handlers decoding the instruction
KLV	Vector subset KLE needs: VS gating, discovery, element widths
Discovery	kl.avail per Machine, or extension names such as Zklaes192pgcmm

Spec v0.6 (Developed, 24 Sep 2026); facts as of c6ada5683d69 (13 Oct 2026).

Backup · Sub-extensions

OpenSSL CONFERENCE

Extension	Adds
`Zklv`	Vector-register operands (needs the KLV subset, VLEN ≥ 128)
`Zklio`	Streaming instructions and the KLIOBUF buffer
`Zklmem`	kl.load / kl.store (hardware or trap-and-emulate)
`Zklmv`	kl.mvin / kl.mvout
`Zklexpire`	ExpirationDate, with a secure clock
`Smklstatus / Ssklstatus`	Per-locker status CSRs: Dirty-only save, lazy restore
`Smklcsk`	The mklcsk CSR group (Context Sealing Key)
`Smklpbootscrt / Ssklvbootscrt`	Physical / virtual boot secrets
`Smkllclt / Sskllclt`	Software Locality CSR groups

Zkl = Zklv or Zklio + Zklmem + AES ECB/GCM, AES-128/256 GCM-SIV, SHA-2.

Backup · Prototype allocations (not spec)

OpenSSL CONFERENCE

Item	Prototype value
Opcodes	The spec's provisional ones: MISC-MEM funct3 3 and 4; kl.load in LOAD funct3 7, kl.store in STORE funct3 5
misa.L, KLS	Bit 11; mstatus / sstatus[26:25]
CSRs	klstart `0x800`, klnamedlocker `0x801`, ID `0xCC0–0xCC3`, skllocality `0x5E0/1`, sllockerstatus `0x5E2`, mklcsk `0xBC0–3`, mllockerstatus `0xBC8`
Causes	24 fatal-error … 29 out-of-memory, 30 RBG-unavailable, 31 clock-unavailable; tval 0
hwprobe	`RISCV_HWPROBE_KEY_IMA_EXT_1`, bit 6
QEMU	`-cpu max,x-zkl=true`, optionally with `x-zkl-csk-random=true`, `x-zkl-ctr=false`, `x-zkl-gcm=false`, `x-zkl-status=false`

Prototype on QEMU, Linux and OpenSSL; spec 9c5ce79 (13 Oct 2026).

Backup · Two ways into OpenSSL

Transparent · default provider

- Raw AES keys go into a locker at key setup: ECB and CBC on the ECB Machine, CTR and GCM on the CTR Machine
- One direction, user mode only, this process
- No application changes; no key schedule in memory for these modes
- Keeps the raw key, to go on in software when no locker is free — at setup or mid-stream

Opaque · kle provider + EVP_SKEY

- **`EVP_SKEY`** opaque keys (OpenSSL 3.5+): generated in the unit, or imported from an SCC a trusted component provisioned
- Exported only as **`kle-scc`**; **`EVP_SKEY_get0_raw_key()`** fails: no raw key exists
- Binding: device (default) or process; QEMU's CSK is random per power-on, so SCCs end with the VM
- Not yet: the TLS chain below — libssl, the KDFs and GCM still pass raw keys — nor **`EVP_PKEY`** keys in lockers

```
/* What TLS 1.3 will need on this route: the read key derived, restricted and used, never in memory. */
EVP_SKEY *ss = EVP_PKEY_derive_SKEY(kx, NULL, "GENERIC-SECRET", "provider=kle", 0, NULL);  /* ECDHE: kl.derive */
EVP_KDF_CTX_set_SKEY(kdf, ss, NULL);          /* TLS13-KDF keyed inside; a PSK would arrive as "kle-scc" */
EVP_SKEY *rk = EVP_KDF_derive_SKEY(kdf, NULL, "AES", "provider=kle", 16, dec);  /* AES-128-GCM, decrypt-only */
EVP_CipherInit_SKEY(ctx, EVP_CIPHER_fetch(libctx, "AES-128-GCM", "provider=kle"), rk, nonce, 12, 0, NULL);
EVP_CipherUpdate(ctx, NULL, &n, hdr, 5);  EVP_CipherUpdate(ctx, pt, &n, rec, len);   /* AAD, then the record */
EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_AEAD_SET_TAG, 16, tag);   /* the received tag */
ok = EVP_CipherFinal_ex(ctx, pt + n, &m);   /* the verdict: Success or Failure in the locker */
```

Backup · TLS with its keys in lockers

OpenSSL CONFERENCE

The QEMU guest fetches this conference's page over TLS 1.3; the record keys sit in lockers.

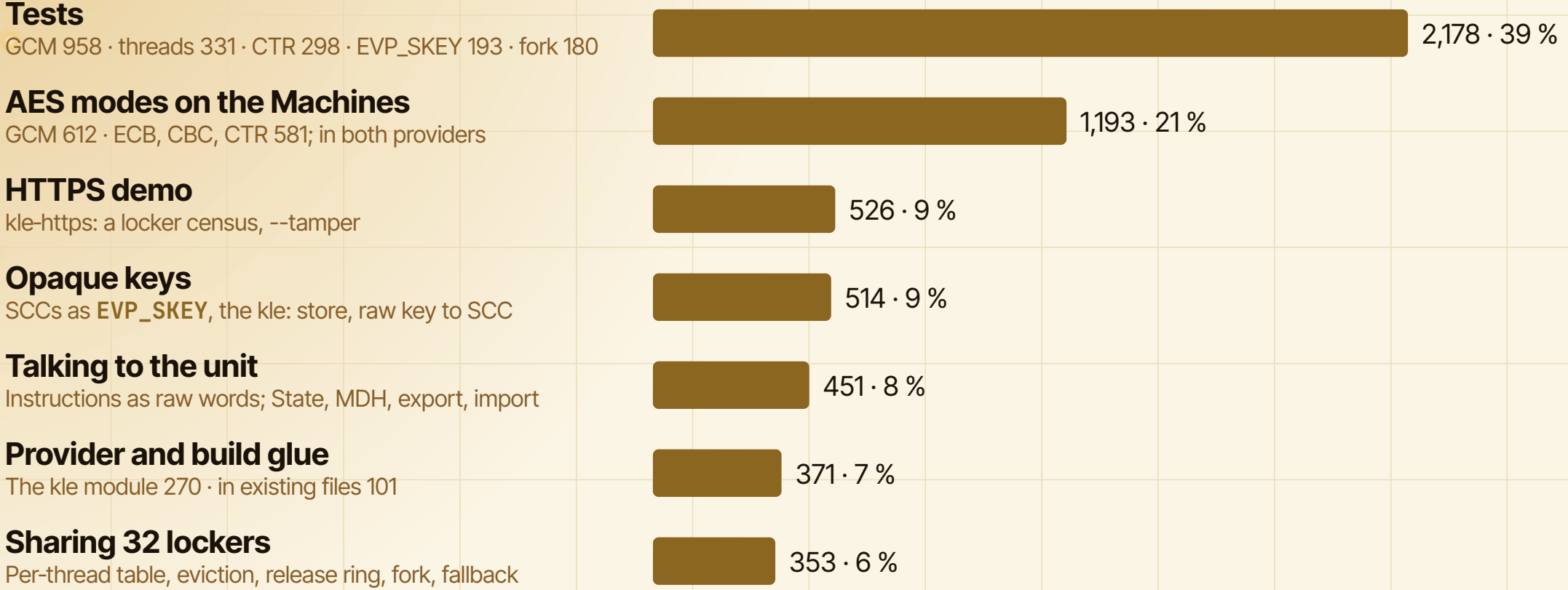
```
$ nix run github:radimkrcmar/kle-demo#https    // the QEMU guest, on its user network
--- kle-https --kle openssl-conference.org
TLSv1.3 TLS_AES_128_GCM_SHA256, records by AES-128-GCM from the kle provider
lockers after the handshake:
  K10 AES128_GCM Ready            dec     kle provider
  K11 AES128_GCM Ready            enc     kle provider
  (9 unchanged since before the connection)    // the DRBGs' AES-256 keys
HTTP/1.0 200 OK, 126434 bytes of body    // then both in Success; none left after SSL_free
--- kle-https --kle --tamper openssl-conference.org
  K10 AES128_GCM Failure          dec     kle provider    // after the failed read: bad record mac
```

K10, the read key, is decrypt-only: the tag's verdict is its State, Success or Failure.

The traffic keys still come from a key schedule in memory; the opaque chain is under “Two ways into OpenSSL”.

Backup · OpenSSL's lines, by purpose

OpenSSL CONFERENCE



Lines OpenSSL **zkl** (dcfcdaa) adds to its upstream parent, by function: 5,586, 5,472 of them in 11 new files; 114 in 14 existing ones.